Laboratorio 4 - Contenedores y Despliegue con Docker

Integrantes: Jefferson Espejo, Mateo Blando
Imagen en Docker Hub: andresespejo/miapp-docker:v1
(https://hub.docker.com/r/andresespejo/miapp-docker)

Descripción técnica

Para el laboratorio desarrollamos una aplicación web pequeña en Node.js usando Express. La app expone tres endpoints HTTP: `/` que devuelve un mensaje de bienvenida, `/sumar?a=5&b=3` que suma dos números y responde en JSON (y devuelve error 400 si los parámetros no son números), y `/salud` que sirve para comprobar que el servicio está funcionando.

Para empaquetarla escribimos un `Dockerfile` basado en la imagen oficial `node:20-alpine`, que es una versión liviana de Linux con Node ya instalado. Primero se copian los archivos `package.json` y `package-lock.json` y se ejecuta `npm install`, y después se copia el resto del código. Lo hicimos en ese orden para que Docker reutilice la capa de dependencias en caché cuando solo cambia el código y así el build sea más rápido. También agregamos un `.dockerignore` para no meter `node_modules` ni archivos innecesarios en la imagen. Finalmente se expone el puerto 3000 y se arranca la app con `node app.js`.

La imagen se construyó con `docker build -t miapp-docker:v1 .` y se ejecutó con `docker run -d -p 3000:3000 miapp-docker:v1`, mapeando el puerto del contenedor al del equipo. Probamos los endpoints desde el navegador y Postman. Por último la etiquetamos con nuestro usuario y la subimos a Docker Hub con `docker push`, de modo que cualquiera puede descargarla con `docker pull` y correrla sin instalar Node.

Cómo Docker mejora la arquitectura de despliegue

- **Portabilidad:** la imagen lleva todo lo que la app necesita (sistema base, Node, dependencias y código), por lo que funciona igual en nuestro computador, en el de otra persona o en un servidor. Se acaba el problema de "en mi máquina sí funciona".
- **Aislamiento:** cada contenedor corre con sus propios procesos, archivos y red, sin chocar con otras aplicaciones o versiones instaladas en el equipo.
- **Reproducibilidad:** el Dockerfile documenta paso a paso cómo se construye el entorno, y al etiquetar versiones (`v1`, `v2`) se puede volver a una versión anterior fácilmente.
- **Escalabilidad:** como los contenedores son livianos y arrancan en segundos, se pueden levantar varias instancias de la misma imagen detrás de un balanceador o con orquestadores como Docker Compose o Kubernetes.
- **Distribución:** con Docker Hub el despliegue se reduce a hacer `pull` y `run`, lo que facilita integrarlo con pipelines de CI/CD.

Conclusión

Con este laboratorio entendimos la diferencia entre una imagen (la plantilla) y un contenedor (la instancia en ejecución), y vimos cómo Docker simplifica empaquetar y desplegar una aplicación de forma consistente en cualquier entorno.